Docker → Kubernetes voor de CKA

Oct 15, 2026 · @Tobias

Docker → Kubernetes: concepten

Vrijwel elk Docker-begrip heeft een Kubernetes-tegenhanger, maar het gedrag verschilt. Docker voert één keer uit wat jij zegt. Kubernetes legt vast wat je wílt (de *desired state*), en controllers blijven daar continu naartoe sturen. Commando's staan hieronder voluit, zonder de alias `k` en zonder afgekorte flags.

Kritische correcties op de vorige versie: (1) container-herstart regelt de **kubelet** via `restartPolicy`, niet de Deployment. De Deployment vervangt hele Pods, bijvoorbeeld bij node-uitval. (2) Docker herstart een *unhealthy* container niet: de restart policy kijkt alleen naar de exitcode. (3) Docker-DNS op containernaam werkt alleen op zelf aangemaakte netwerken, niet op de standaard bridge. (4) `ENTRYPOINT` / `CMD` ontbrak, en is een klassieke valkuil. (5) Labels zijn in Kubernetes geen metadata maar de lijm tussen objecten.

1. Draaien, herstarten en schalen

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker run --detach --name web nginx:1.27`	Start één container op deze host, op de achtergrond	`kubectl run web --image=nginx:1.27`	Maakt één Pod; de scheduler kiest een node, de kubelet daar start de container	Een Pod is de kleinste eenheid: 1+ containers die netwerk (localhost) en volumes kunnen delen
`docker run --detach --restart=always --name web nginx:1.27`	De Docker-daemon start de container opnieuw als hij stopt (op basis van exitcode)	Standaard: `restartPolicy: Always` in de Pod-spec (geen apart commando)	De **kubelet** herstart de container op dezelfde node, met wachttijd 10 s, 20 s, 40 s… tot max. 5 min (= `CrashLoopBackOff`)	Valt de node uit, dan doet een losse Pod niets. Een Pod wordt nooit naar een andere node verplaatst; een controller maakt een nieuwe

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker compose up --detach --scale web=3`	Start 3 containers van dezelfde service op deze host	`kubectl create deployment web --image=nginx:1.27 --replicas=3`	Maakt een Deployment → ReplicaSet → 3 Pods, verspreid over nodes; verdwijnt er één, dan maakt de ReplicaSet een nieuwe	Docker schaalt één keer; Kubernetes houdt het aantal continu vast. Later aanpassen: `kubectl scale deployment web --replicas=5`
`docker service create --mode global --name agent mijn-agent:1.0` (Swarm)	Eén container op elke Swarm-node	Geen imperatief commando: genereer YAML met `kubectl create deployment agent --image=mijn-agent:1.0 --dry-run=client --output=yaml`, verander `kind` in `DaemonSet` en verwijder `replicas` en `strategy`	Eén Pod op elke (passende) node, ook op nodes die later bijkomen	Examen-truc: DaemonSet en StatefulSet kun je niet met `kubectl create` maken
`docker run --rm busybox:1.36 echo hallo`	Draait één keer, ruimt de container daarna op	`kubectl create job hallo --image=busybox:1.36 -- echo hallo`	Een Job draait Pods tot ze succesvol eindigen (met herpogingen)	Interactief en tijdelijk: `kubectl run tmp --image=busybox:1.36 --rm --stdin --tty --restart=Never -- sh`
Host-cron: `0 2 * * * docker run --rm backup:1.0`	De crontab van de host start periodiek een container	`kubectl create cronjob backup --image=backup:1.0 --schedule="0 2 * * *"`	Een CronJob maakt volgens schema Jobs aan, in het cluster zelf	Geen afhankelijkheid van één host; historie en mislukte runs zie je met `kubectl get jobs`
Container met vaste naam + eigen volume	Handmatig: `docker run --name db-0 --volume db0:/data postgres:17`	Geen imperatief commando: StatefulSet in YAML	Pods met vaste namen (`db-0`, `db-1`), vaste volgorde en per Pod een eigen PVC	Voor databases en clusters met vaste identiteit

2. Starten en configureren

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker run --entrypoint /bin/sh busybox:1.36 -c "echo hallo"`	`--entrypoint` vervangt `ENTRYPOINT`; wat na het image staat vervangt `CMD`	`kubectl run hallo --image=busybox:1.36 --command -- /bin/sh -c "echo hallo"`	Met `--command` wordt alles na `--` het **command** (= `ENTRYPOINT`); zonder `--command` worden het **args** (= `CMD`)	Valkuil: `command` ↔ `ENTRYPOINT`, `args` ↔ `CMD`. Geef je alleen `args`, dan blijft het standaard-command van het image staan
`docker run --env MODE=prod --env-file ./app.env nginx:1.27`	Zet omgevingsvariabelen in die ene container	`kubectl create configmap app-config --from-literal=MODE=prod --from-env-file=./app.env` en daarna `kubectl set env deployment/web --from=configmap/app-config`	De ConfigMap is een apart object; de Deployment haalt er variabelen uit (of mount het als bestanden)	Config staat los van de workload en kan door meerdere Pods gedeeld worden. Wijzigen van env-vars vraagt een herstart van de Pods
`docker secret create db_pw ./pw.txt` (Swarm)	Versleuteld opgeslagen in de Swarm, als bestand gemount	`kubectl create secret generic db-pw --from-file=password=./pw.txt`	Een Secret, te gebruiken als env-var of als gemount bestand	Een Secret is standaard alleen **base64**, niet versleuteld in etcd (tenzij je encryption at rest instelt). Bescherm met RBAC
`docker run --memory=256m --cpus=0.5 nginx:1.27`	Harde grenzen voor deze container op deze host	`kubectl set resources deployment web --requests=cpu=250m,memory=128Mi --limits=cpu=500m,memory=256Mi`	**Requests** = wat de scheduler reserveert om een node te kiezen. **Limits** = harde grens; boven het geheugenlimiet volgt `OOMKilled`	Docker heeft geen scheduler, dus geen "requests". Zonder requests kan Kubernetes nodes overvol plannen
`docker run --user 1000:1000 --read-only --cap-drop ALL nginx:1.27`	Draait als niet-root, read-only filesystem, zonder Linux-capabilities	Geen imperatief commando: `securityContext` in de YAML (`runAsUser: 1000`, `readOnlyRootFilesystem: true`, `capabilities.drop: ["ALL"]`)	Zelfde beperkingen, per Pod of per container instelbaar	Met Pod Security Admission kun je dit per namespace afdwingen (CKS-stof)
`docker run --health-cmd "curl -f http://localhost/" --health-interval 10s nginx:1.27`	Docker zet de status op `healthy` / `unhealthy`; standalone Docker **herstart niet** bij unhealthy	Geen imperatief commando: probes in de YAML (`livenessProbe`, `readinessProbe`, `startupProbe`)	**liveness** faalt → kubelet herstart de container. **readiness** faalt → Pod gaat uit de Service (geen verkeer). **startup** → geeft trage apps tijd	Drie soorten met elk een eigen gevolg, en ze doen echt iets

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`depends_on: [db]` (compose)	Compose start eerst `db`, dan de app	`initContainers` in de YAML, bijvoorbeeld `until nslookup db; do sleep 2; done`	Init-containers draaien na elkaar en moeten slagen vóór de app-containers in dezelfde Pod starten	Tussen Pods bestaat geen startvolgorde; je app moet zelf tegen een ontbrekende dependency kunnen

3. Netwerk

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker network create appnet` + `docker run --network appnet …`	Maakt een geïsoleerd bridge-netwerk; alleen containers op dat netwerk zien elkaar	Geen commando: de CNI-plugin maakt één plat netwerk voor het hele cluster	Elke Pod krijgt een eigen IP en kan **elke** andere Pod bereiken, ook op andere nodes	Isoleren doe je achteraf met NetworkPolicies: `kubectl apply --filename=default-deny.yaml` (vereist een CNI die ze afdwingt, zoals Calico)
Containernaam als DNS-naam (`ping db`)	Werkt alleen op zelf aangemaakte netwerken, niet op de standaard bridge	Service-naam: `db`, of voluit `db.<namespace>.svc.cluster.local`	CoreDNS vertaalt **Service**-namen naar het stabiele ClusterIP	Pods vind je niet op naam maar via een Service; Pod-IP's veranderen bij elke herstart
`docker run --publish 8080:80 nginx:1.27`	Koppelt poort 8080 van de host aan poort 80 van de container	`kubectl expose deployment web --port=80 --target-port=80 --type=NodePort`	Maakt een Service met een vast ClusterIP; NodePort opent een poort (30000–32767) op **elke** node die naar de Pods doorstuurt	`containerPort` in de Pod-spec publiceert niets. Tijdelijk testen: `kubectl port-forward deployment/web 8080:80`
Reverse proxy (Traefik/nginx) met host-regels	Een gewone container die op hostnaam doorstuurt	`kubectl create ingress web --class=nginx --rule="web.lab/*=web:80"`	Ingress-object met routeringsregels; een **Ingress-controller** voert ze uit. Opvolger: Gateway API (`Gateway` + `HTTPRoute`)	Zonder controller doet het Ingress-object helemaal niets

4. Opslag

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker volume create data` + `docker run --volume data:/data postgres:17`	Een door Docker beheerd volume op deze host	Geen imperatief commando: PersistentVolumeClaim (PVC) in YAML, gekoppeld aan een StorageClass	De PVC vraagt opslag; een StorageClass maakt automatisch een PersistentVolume (PV) aan, of een beheerder maakt de PV vooraf	Extra lagen: **accessModes** (RWO/RWX), **reclaimPolicy** (Retain/Delete). Data kan meeverhuizen tussen nodes als de storage dat ondersteunt
`docker run --volume /srv/html:/usr/share/nginx/html:ro nginx:1.27`	Bind mount van een map op de host	`hostPath`-volume in de Pod-YAML	Mount een map van de node waar de Pod toevallig draait	Gevaarlijk buiten labs: komt de Pod op een andere node, dan zie je andere (of geen) data
`docker run --tmpfs /cache nginx:1.27`	Tijdelijk geheugenfilesystem	`emptyDir`-volume (met `medium: Memory` voor RAM)	Lege map die leeft zolang de Pod bestaat; standaard op de disk van de node	Overleeft een container-herstart, niet het verwijderen van de Pod. In RAM telt het mee voor de geheugenlimiet

5. Organiseren en declaratief werken

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker compose up --detach`	Start alles uit `compose.yaml`	`kubectl apply --filename=./app/`	Maakt of werkt alle manifests in de map bij; meerdere keren draaien is veilig	Templating/hergebruik: Helm (`helm install`) of Kustomize (`kubectl apply --kustomize=./overlay`)
`docker compose down`	Stopt en verwijdert de containers	`kubectl delete --filename=./app/`	Verwijdert alle objecten uit die manifests	PVC's verwijderen kan data meenemen, afhankelijk van de reclaimPolicy
`docker compose --project-name shop up --detach`	Zet de projectnaam als prefix voor containers en netwerken	`kubectl create namespace shop` en `kubectl apply --filename=./app/ --namespace=shop`	Een namespace scheidt namen, quota en rechten (RBAC)	Namespaces scheiden **niet** het netwerk: daarvoor heb je NetworkPolicies

Docker-commando	Wat doet het (Docker)	Kubernetes-commando	Wat doet het (Kubernetes)	Verschil
`docker run --label tier=web nginx:1.27`	Metadata op de container, vooral voor tools (bijv. Traefik)	`kubectl label pod web tier=web`	Labels zijn functioneel: Services, Deployments en NetworkPolicies vinden Pods via label-selectors	Klopt een label niet, dan heeft een Service geen endpoints: een klassieke troubleshooting-vraag
`docker pull nginx:1.27`	Haalt het image vooraf binnen	Geen commando: de kubelet haalt het image zelf op bij het starten van een Pod	`imagePullPolicy`: standaard `IfNotPresent`, maar `Always` bij tag `:latest` of zonder tag	Elke node heeft een eigen image-cache; private registries via een `imagePullSecret`
`docker info`	Info over de lokale Docker-daemon	`kubectl get nodes --output=wide` (kolom CONTAINER-RUNTIME) en op de node `crictl info`	Toont per node welke runtime draait (containerd via CRI)	Op Kubernetes-nodes staat geen Docker: de kubelet praat via de CRI met containerd

Docker CLI → kubectl

Docker praat met één lokale daemon; `kubectl` praat met de API-server van het cluster dat in je kubeconfig (`~/.kube/config`) als context is gekozen. Je beschrijft wat je wilt, en controllers in het cluster voeren het uit. Op het examen is `k` al een alias voor `kubectl` met bash-completion; k9s heb je daar niet.

Zo lees je een kubectl-commando

`kubectl <werkwoord> <resourcetype> [naam] [-n namespace] [flags]`

- **Werkwoord**: wat je doet — `get` (lijst), `describe` (samenvatting + events), `create` / `apply` (aanmaken), `edit`, `delete`, `logs`, `exec`.
- **Resourcetype**: welk soort object — `pod` / `po`, `deployment` / `deploy`, `service` / `svc`, `node` / `no`, `namespace` / `ns`, `configmap` / `cm`. Alle types en afkortingen: `k api-resources`.
- **Namespace**: de meeste objecten leven in een namespace. Zonder `-n` werk je in `default`; `-A` = alle namespaces.

- **Uitvoer**: `-o wide` (extra kolommen), `-o yaml` (volledig object), `-o name` (alleen namen, handig in scripts).

Docker CLI naast kubectl

Docker	kubectl	Wat het doet	Verschil / let op
`docker ps`	`k get po -A -o wide`	Toont alle Pods in alle namespaces, met de node en het Pod-IP erbij	Je ziet Pods, geen losse containers. `READY 1/2` = 1 van 2 containers klaar; `RESTARTS` = hoe vaak een container crashte
`docker ps -a`	`k get po -A --field-selector=status.phase!=Running`	Filtert op Pods die niet draaien (Pending, Failed, Succeeded)	`--field-selector` filtert op velden van het object; op labels filter je met `-l app=web`
`docker run -d --name web nginx`	`k run web --image=nginx`	Maakt één losse Pod met één container	Niemand maakt hem opnieuw als de node uitvalt. Voor echte apps gebruik je een Deployment
`docker run` met `--scale 3`	`k create deploy web --image=nginx --replicas=3`	Maakt een Deployment, die een ReplicaSet maakt, die 3 Pods in stand houdt	Verwijder je een Pod, dan maakt de controller direct een nieuwe. Opschalen: `k scale deploy web --replicas=5`
`docker compose up -d`	`k apply -f app.yaml`	Maakt of werkt objecten bij volgens het bestand	`create` faalt als het object al bestaat; `apply` maakt aan óf werkt bij — je kunt het herhalen
`docker run -p 8080:80`	`k expose deploy web --port=80 --type=NodePort`	Maakt een Service die verkeer verdeelt over de Pods met de labels van de Deployment; NodePort opent een poort (30000–32767) op elke node	`containerPort` in een Pod publiceert niets. Tijdelijk vanaf je terminal: `k port-forward deploy/web 8080:80` (stopt als het commando stopt)
Nieuwe tag: container opnieuw starten	`k set image deploy/web nginx=nginx:1.27`	Start een rolling update: nieuwe Pods op, oude af, zonder downtime	Terug: `k rollout undo deploy/web` ; volgen: `k rollout status deploy/web`

Docker	kubectl	Wat het doet	Verschil / let op
`docker logs -f web`	`k logs -f deploy/web`	Volgt stdout/stderr van een container; `deploy/web` kiest zelf een Pod van de Deployment	`--previous` toont de logs van de vorige, gecrashte container (onmisbaar bij CrashLoopBackOff). Meerdere containers in de Pod: `-c <naam>`
`docker exec -it web sh`	`k exec -it web -- sh`	Start een shell in de container	De `--` scheidt kubectl-flags van het commando in de container. Image zonder shell: `k debug`
`docker inspect web`	`k describe po web` / `k get po web -o yaml`	`describe`: leesbare samenvatting met onderaan **Events** (scheduling, image pull, probes). `-o yaml`: het volledige object zoals het in de API staat	Events zijn je eerste stop bij troubleshooting: daar staat waarom een Pod niet start
`docker stop` / `docker rm`	`k delete po web`	Verwijdert het object; de container krijgt eerst tijd om netjes te stoppen (standaard 30 s)	Een Pod van een Deployment komt meteen terug: verwijder dan de Deployment. Direct weg: `--force --grace-period=0`
`docker cp`	`k cp web:/pad ./lokaal`	Kopieert bestanden tussen container en je machine	Werkt via `tar`, dus `tar` moet in het image zitten
`docker stats`	`k top po` / `k top no`	Toont CPU- en geheugengebruik per Pod of node	Werkt alleen met metrics-server; anders: "Metrics API not available"
`docker events`	`k get events -A --sort-by=.lastTimestamp`	Toont clustergebeurtenissen op tijdsvolgorde	Events worden na korte tijd opgeruimd, dus kijk snel
`docker images`	`crictl images` (op de node zelf)	Toont de images die containerd op die node heeft	Er is geen centrale imagelijst: elke node heeft een eigen cache
`docker ps` op de host	`crictl ps -a` / `crictl logs <id>`	Praat rechtstreeks met containerd, buiten Kubernetes om	Werkt ook als de API-server plat ligt — zo debug je een kapotte control plane

Docker	kubectl	Wat het doet	Verschil / let op
`docker compose config`	`k apply --dry-run=server -f x.yaml` / `k diff -f x.yaml`	`--dry-run=server`: de API-server valideert alles zonder op te slaan. `diff`: verschil tussen cluster en bestand	`--dry-run=client` checkt alleen lokaal en mist fouten die de server pas ziet
`systemctl status docker`	`systemctl status kubelet` + `journalctl -u kubelet`	Status en logs van de kubelet, de agent die op elke node de Pods start	Eerste stap bij een NotReady node
`docker context use`	`k config use-context <ctx>`	Wisselt naar een ander cluster/gebruiker uit je kubeconfig	Elke examenvraag begint met een context-commando: altijd eerst uitvoeren. Standaard-namespace zetten: `k config set-context --current --namespace=x`
`docker network ls` / `docker volume ls`	`k get svc,ep,netpol` / `k get pv,pvc,sc`	Netwerk- en opslagobjecten bekijken	Geen één-op-één: netwerk = Services + NetworkPolicies, opslag = PV/PVC/StorageClass
`docker build` / `docker pull`	—	Geen kubectl-equivalent	Images bouw je buiten het cluster; de kubelet trekt ze zelf binnen bij het starten

YAML laten genereren in plaats van typen

De belangrijkste examen-oneliner:

```
export do="--dry-run=client -o yaml"; k create deploy web --image=nginx $do > web.yaml
```

- `--dry-run=client` laat kubectl het object alleen lokaal opbouwen; er gaat niets naar het cluster.
- `-o yaml` print dat object als YAML; `> web.yaml` zet het in een bestand.
- Daarna pas je `web.yaml` aan (bijvoorbeeld probes of resources toevoegen) en doe je `k apply -f web.yaml`.
- `export do=…` bewaart de flags in een variabele, zodat je alleen `$do` hoeft te typen.

Veldnamen opzoeken zonder de docs

`k explain pod.spec.containers.livenessProbe` toont welke velden bestaan, hun type en uitleg, rechtstreeks uit de API-server. Met `--recursive` zie je de hele boom eronder. Handig als je niet meer weet of het `readinessProbe.httpGet.port` of iets anders was.

Niet vertaalbaar: uniek voor Kubernetes

Dit heeft geen Docker-tegenhanger. Hier zit het grootste deel van je studietijd, omdat je Docker-reflex je hier niet helpt.

Begrip	Wat het is	CKA-domein
Control plane	kube-apiserver, etcd, kube-scheduler, kube-controller-manager; op kubeadm als static Pods in `/etc/kubernetes/manifests`	Cluster Architecture, Troubleshooting
etcd	De volledige clusterstaat; backup en restore met `etcdctl snapshot save/restore`	Cluster Architecture
Reconciliation-loop	Controllers vergelijken gewenst met actueel en sturen bij; je verwijdert een Pod en hij komt terug	Workloads
kubeadm-lifecycle	`kubeadm init/join/upgrade`, certificaten (`kubeadm certs check-expiration`)	Cluster Architecture
Static Pods	Door de kubelet zelf beheerd vanuit een map, buiten de API-server om	Troubleshooting
Scheduling	nodeSelector, node-affinity, taints/tolerations, PriorityClass	Workloads & Scheduling
Node-onderhoud	`cordon`, `drain`, `uncordon`	Cluster Architecture
RBAC	Role/ClusterRole + (Cluster)RoleBinding + ServiceAccount; testen met `k auth can-i --as`	Cluster Architecture
kubeconfig	Clusters, users en contexts; certificaat-gebaseerde identiteit	Cluster Architecture

Begrip	Wat het is	CKA-domein
NetworkPolicy	Firewall tussen Pods op labels; vereist een CNI die het afdwingt (Calico, Cilium)	Services & Networking
Gateway API	Opvolger van Ingress: GatewayClass, Gateway, HTTPRoute	Services & Networking
CRD's en operators	Eigen API-objecten plus een controller die ze uitvoert	Cluster Architecture
Extensie-interfaces	CNI (netwerk), CSI (storage), CRI (runtime) als plug-ins	Cluster Architecture
PV-lifecycle	accessModes (RWO/RWX), reclaimPolicy (Retain/Delete), binding tussen PVC en PV	Storage
Rollouts	`k rollout status/history/undo`, strategy RollingUpdate	Workloads
Resource-governance	ResourceQuota, LimitRange per namespace	Workloads
Admission	Controllers die objecten valideren of aanpassen voor opslag	Cluster Architecture
Endpoints / EndpointSlices	De echte Pod-IP's achter een Service; leeg = selector klopt niet	Troubleshooting

AKS vs on-prem (kubeadm)

Het API-oppervlak (kubectl, manifests, Helm) is identiek. Het verschil is wie de control plane en de infra-plug-ins beheert. Het examen draait op kubeadm-achtige clusters, dus de on-prem kolom is examenstof en de AKS-kolom niet.

Onderdeel	AKS (Azure)	On-prem (kubeadm)	Examenrelevant
Control plane	Beheerd door Azure, niet zichtbaar als Pods	Static Pods op jouw control plane nodes	Ja

Onderdeel	AKS (Azure)	On-prem (kubeadm)	Examenrelevant
etcd	Beheerd, geen toegang	Zelf backuppen en restoren met `etcdctl`	Ja
Cluster aanmaken	`az aks create` / Terraform (azurerm)	`kubeadm init` + `kubeadm join`	Ja (kubeadm)
Upgrades	`az aks upgrade`, node-image-upgrades	`kubeadm upgrade plan/apply` per node + kubelet	Ja (kubeadm)
HA control plane	Inbegrepen; uptime-SLA via Standard-tier	Zelf: 3 control plane nodes + load balancer voor de API	Ja (concept)
Nodes	Node pools op VM Scale Sets (system en user pools)	Eigen VM's of bare metal	Nee
CNI	Azure CNI (overlay), Azure CNI powered by Cilium	Zelf kiezen en installeren: Calico, Cilium, Flannel	Ja (installeren)
`Service type=LoadBalancer`	Werkt direct: Azure Load Balancer via cloud-controller-manager	Blijft `<pending>` zonder MetalLB of vergelijkbaar	Ja (concept)
Storage	CSI-drivers voor Azure Disk/Files, StorageClasses voorgeïnstalleerd	Zelf een CSI-driver of local-path provisioner	Ja (StorageClass/PV)
Identiteit	Entra ID-integratie + Azure RBAC of Kubernetes RBAC	Certificaten in kubeconfig + Kubernetes RBAC	Ja (Kubernetes RBAC)
Kosten	Free-tier zonder SLA; Standard- en Premium-tier betaald	Eigen hardware	Nee
Troubleshooting node	Beperkt (`kubectl debug node/...`), geen SSH standaard	`ssh`, `systemctl`, `journalctl -u kubelet`, `crictl`	Ja

Handig voor je lab: oefen alles uit de rechterkolom op je kubeadm multi-node cluster. Gebruik AKS alleen om LoadBalancer en CSI "werkend" te zien en de verschillen te begrijpen.

CKA study goals

CKA op Kubernetes v1.35, 2 uur, praktijkexamen. Troubleshooting (30%) en Cluster Architecture (25%) zijn samen 55%: begin daar. Status vooringevuld op wat je al gedaan hebt; zet zelf om.

Domein (gewicht)	Doel	Docker-brug	Status
Troubleshooting (30%)	NotReady node fixen: kubelet, `journalctl`, certs	`systemctl status docker`	Nog niet
Troubleshooting (30%)	Kapotte control plane static Pod herstellen	Geen	Nog niet
Troubleshooting (30%)	CrashLoopBackOff, ImagePullBackOff, OOMKilled diagnosticeren	`docker logs`, exit codes	Nog niet
Troubleshooting (30%)	Service zonder Endpoints, DNS-problemen	Compose-servicenaam resolven	Nog niet
Cluster Architecture (25%)	Cluster bouwen met `kubeadm init/join` + CNI	Geen	Bezig
Cluster Architecture (25%)	Upgrade een minor versie met kubeadm	Geen	Nog niet
Cluster Architecture (25%)	etcd snapshot save en restore	Volume-backup	Nog niet
Cluster Architecture (25%)	RBAC: Role, Binding, ServiceAccount, `auth can-i`	`--user`, groepen	Nog niet
Cluster Architecture (25%)	Helm en Kustomize installeren/patchen	Compose + override-file	Bezig
Cluster Architecture (25%)	CRD's en operators installeren	Geen	Nog niet
Services & Networking (20%)	ClusterIP, NodePort, LoadBalancer	`-p 8080:80`	Nog niet

Domein (gewicht)	Doel	Docker-brug	Status
Services & Networking (20%)	NetworkPolicy default-deny + allow	Aparte docker networks	Nog niet
Services & Networking (20%)	Gateway API (HTTPRoute) en Ingress	Traefik/nginx-proxy	Nog niet
Services & Networking (20%)	CoreDNS begrijpen en debuggen	Embedded DNS	Nog niet
Workloads & Scheduling (15%)	Deployment, rollout en rollback	`compose up` met nieuwe tag	Nog niet
Workloads & Scheduling (15%)	ConfigMaps en Secrets	`.env`, secrets	Nog niet
Workloads & Scheduling (15%)	HPA en requests/limits	`--cpus`, `--memory`	Nog niet
Workloads & Scheduling (15%)	Affinity, taints/tolerations	Geen	Nog niet
Storage (10%)	StorageClass, PV, PVC, accessModes, reclaimPolicy	Named volumes	Nog niet
Algemeen	Snelheid met `kubectl` zonder k9s, `--dry-run=client -o yaml`	Docker CLI	Bezig

AWX via Helm telt al als oefening voor Helm; meer heb je niet bewezen afgevinkt. Na een paar doelen: doe de twee killer.sh-sessies die bij je examen horen.

Bronnen

Domeingewichten en v1.35 zijn bevestigd door de officiële LF-pagina, het CNCF-curriculum en meerdere studiegidsen. Dat k9s ontbreekt staat niet letterlijk in de LF-docs: zij noemen alleen terminal, VSCodium en Firefox. Check daarom de officiële "Resources Allowed"-pagina vlak voor je examen.

- Linux Foundation: CKA-pagina (domeinen, v1.35, 2 uur)
- CNCF curriculum (CKA_Curriculum_v1.35)
- LF Handbook: ExamUI voor praktijkexamens
- Open Exam Prep: examenomgeving (alias k, completion, toegestane docs)
- KodeKloud community: geen third-party tools op het examen
- Microsoft Learn: AKS core concepts (beheerde control plane, node pools, tiers)

Bronnen bij de concepttabel (opgehaald 15-10-2026):

- Kubernetes: command en args (ENTRYPOINT/CMD)
- Kubernetes: Pod lifecycle (restartPolicy, back-off, geen herplanning)
- Docker: restart policies (op basis van exitcode)
- Docker: networking (DNS op naam alleen op user-defined netwerken)